Seguridad y Calidad en el Desarrollo (CDY2203)

Informe escrito

IGNACIO KRIMAN BRAVO

20/04/2026

Índice

Investigación de herramientas de análisis SCA……3
Selección de Herramienta……..4
Configuración Base de Datos……....6
Resultados del Análisis de Composición de Software (SCA)……...7
Implementación de Mejoras y Correcciones……9

Investigación de herramientas de análisis SCA

Herramienta	*Organización*	*Tipo*	*Características Principales*
OWASP Dependency Check	OWASP Foundation	Software Libre	Identifica dependencias con vulnerabilidades conocidas (CVE). Se integra nativamente con Maven y genera reportes detallados en HTML.
Snyk	Snyk Ltd.	Comercial (Freemium)	Ofrece escaneo en tiempo real, integración con repositorios Git (GitHub/GitLab) y guías de remediación automática.
Sonatype Nexus	Sonatype	Comercial	Permite un control estricto sobre el ciclo de vida de los componentes y el cumplimiento de políticas de licencias en empresas.

Figura 1: Tabla comparativa de herramientas de análisis SCA

Características Principales de las Herramientas SCA

1. **OWASP Dependency Check:**
    - **Motor de análisis:** Utiliza el identificador de evidencia (CPE) para contrastar librerías contra la base de datos nacional de vulnerabilidades (NVD).
    - **Integración:** Se ejecuta como un plugin de Maven, lo que permite realizar escaneos locales sin subir el código a la nube.
    - **Link:** https://owasp.org/www-project-dependency-check/
2. **Snyk:**
    - **Base de Datos Propia:** Cuenta con una base de datos de vulnerabilidades (Snyk Intel) que suele ser más rápida en reportar fallos que los organismos gubernamentales.
    - **Remediación:** Ofrece consejos específicos de "Upgrade" (actualización) indicando exactamente qué versión instalar para corregir el fallo.
    - **Link:** https://snyk.io/
3. **Sonatype Nexus Lifecycle:**

- **Control de Políticas:** Permite definir reglas automáticas (ej: bloquear cualquier librería con vulnerabilidad Crítica) en empresas con muchos proyectos.
- **Visibilidad:** Ofrece un gráfico detallado de dónde se usa cada componente en toda la organización.
- **Link:** https://www.sonatype.com/products/lifecycle

Selección de Herramienta

Para este proyecto se seleccionó **OWASP Dependency Check**. La razón principal es su naturaleza de código abierto y su capacidad de integración directa en el flujo de construcción de Maven mediante un plugin. Al ser una herramienta estándar recomendada por OWASP, garantiza una base de datos de vulnerabilidades actualizada y confiable para un entorno de desarrollo profesional.

Originalmente, el análisis se planteó utilizando *OWASP Dependency Check*. Sin embargo, durante la fase de ejecución, se presentaron problemas de latencia y bloqueos de red (Rate Limiting) en los servidores del NVD, lo que impidió la sincronización de la base de datos de vulnerabilidades local.

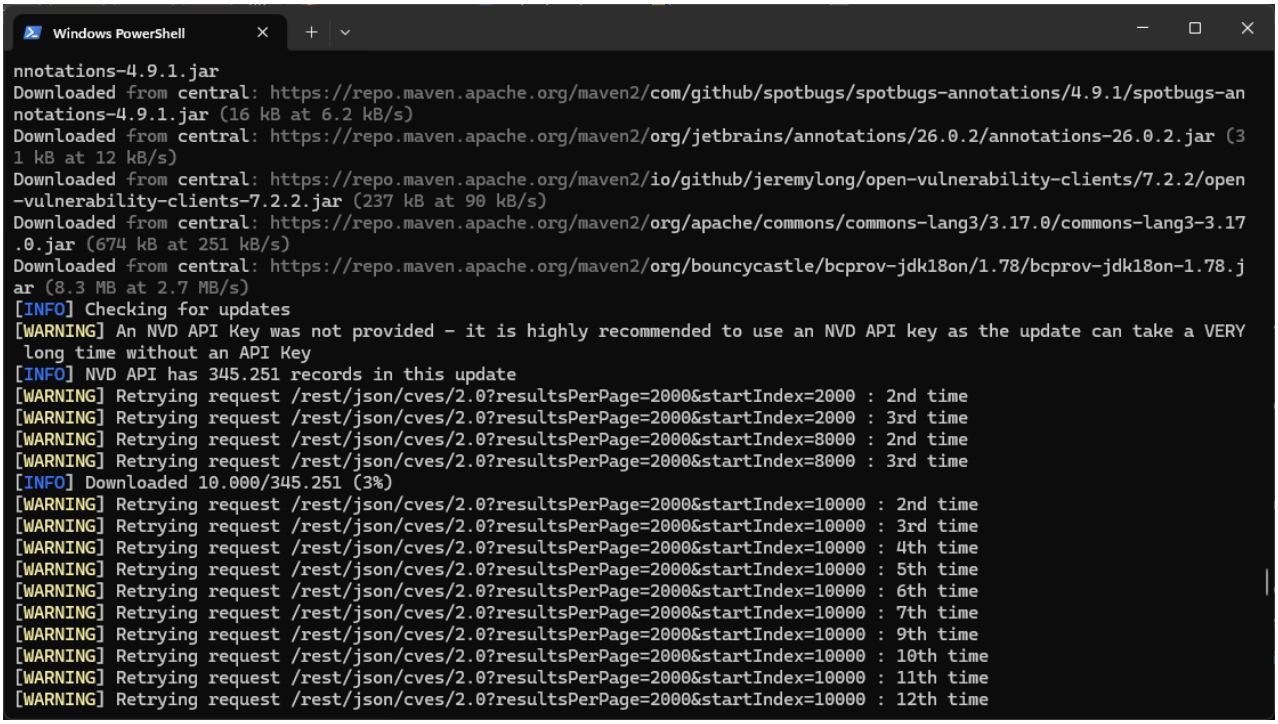
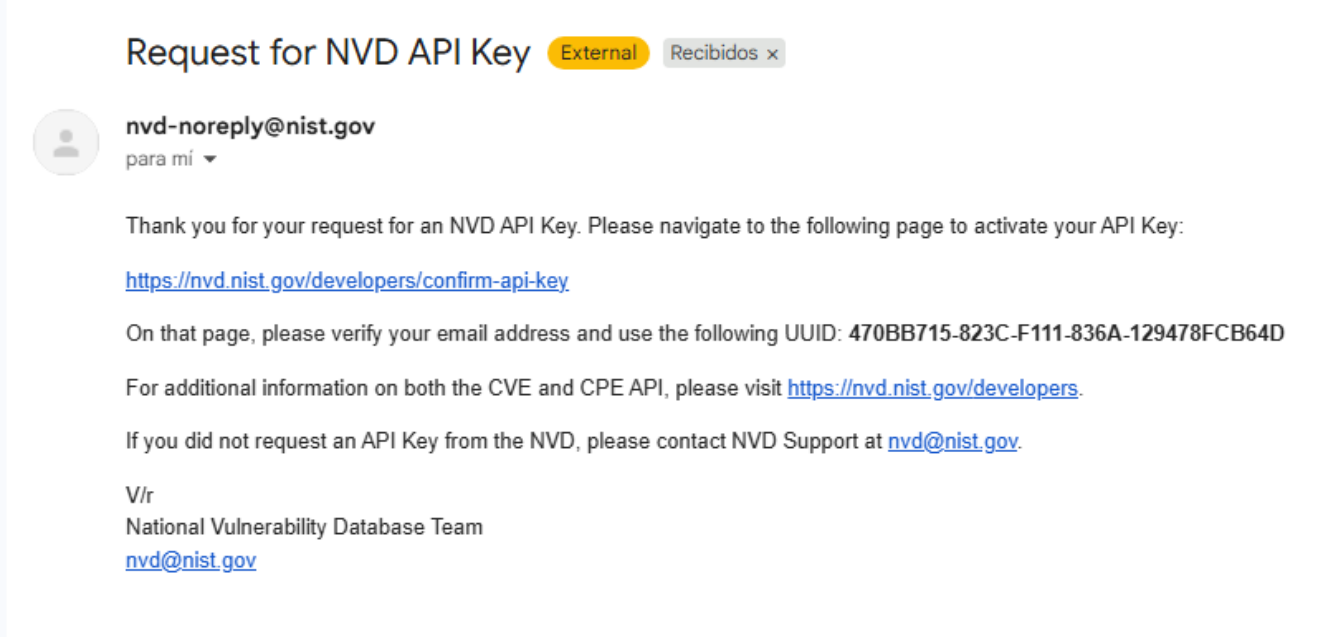


Figura 2: Terminal PowerShell intentando conectar con el NVD.

Se solicito una clave única de acceso para mejorar el ingreso a las dependencias de NVD pero esto no tuvo éxito alguno.



Request for NVD API Key External Recibidos ×

nvd-noreply@nist.gov
para mí

Thank you for your request for an NVD API Key. Please navigate to the following page to activate your API Key:

https://nvd.nist.gov/developers/confirm-api-key

On that page, please verify your email address and use the following UUID: **470BB715-823C-F111-836A-129478FCB64D**

For additional information on both the CVE and CPE API, please visit https://nvd.nist.gov/developers.

If you did not request an API Key from the NVD, please contact NVD Support at nvd@nist.gov.

V/r
National Vulnerability Database Team
nvd@nist.gov

Figura 3: Correo con el API Key para mejor conectividad con la NVD

Ante esta contingencia, se decidió migrar el análisis a **Snyk**. Las razones principales fueron:

1. **Disponibilidad:** Al ser una plataforma basada en la nube, no depende de descargas locales de bases de datos pesadas.
2. **Precisión:** Snyk proporcionó un desglose más detallado de las vulnerabilidades transitivas (aquellas que traen las librerías de Spring Boot), facilitando la identificación de fallos críticos en Tomcat y Spring Security que otras herramientas no mostraron con tanta claridad.
3. **Eficiencia en la Corrección:** La capacidad de Snyk para indicar la versión exacta de parcheo permitió realizar el 'Update' en el archivo pom.xml de forma inmediata y segura."

Configuración Base de Datos

Para la persistencia de datos de la aplicación, se configuró un contenedor Docker con MySQL 8.0. La conexión entre el backend y la base de datos se gestiona a través del archivo src/main/resources/application.properties. En este archivo se especificó la URL de conexión jdbc:mysql://localhost:3306/veterinarias6, junto con las credenciales de usuario y el driver correspondiente, permitiendo que Spring Data JPA gestione las entidades de forma automática.

Codigo referencial utilizado:

```
spring.datasource.url=jdbc:mysql://localhost:3306/veterinarias6?useSSL=false&serverTimezone=UTC
spring.datasource.username=user
spring.datasource.password=user
```

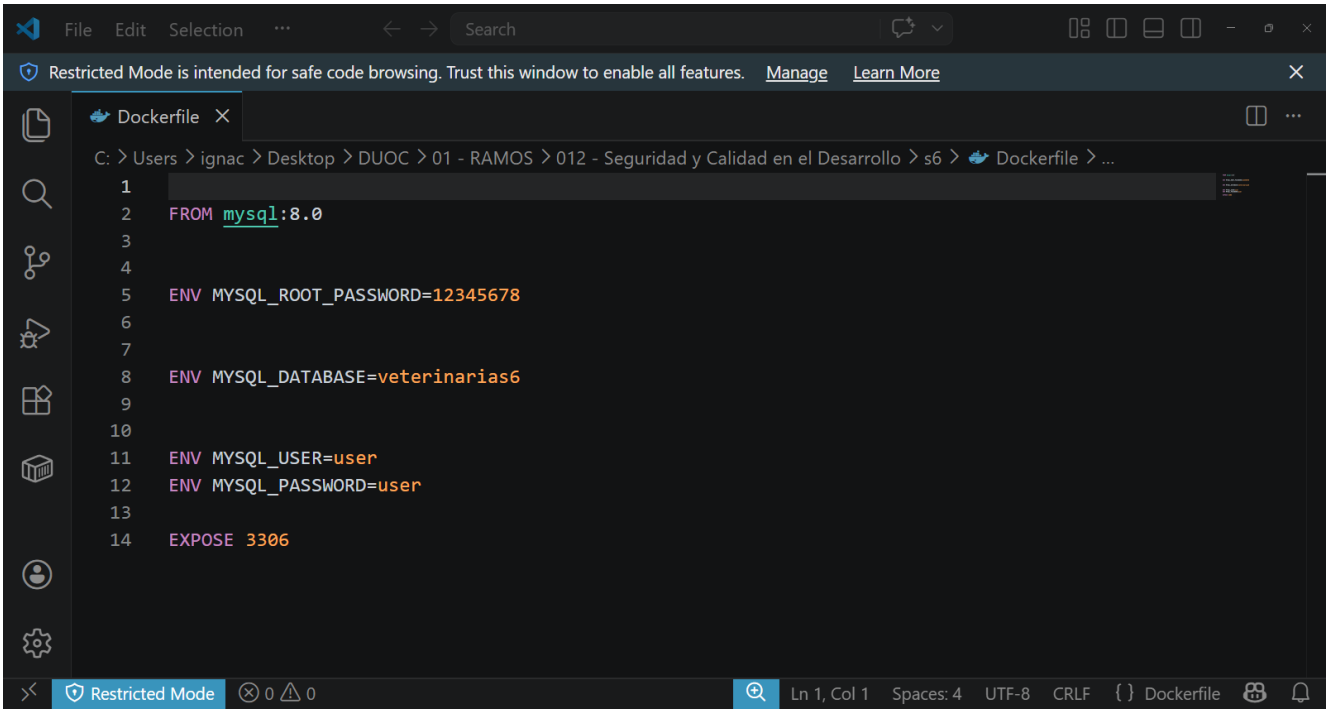


Figura 4: VSC con el archivo DockerFile conteniendo la base de datos.

Ejecución de la herramienta SCA

Dentro de Snyk y mediante Github, se cargo el Pom.xml del proyecto para su análisis detallado de vulnerabilidades.

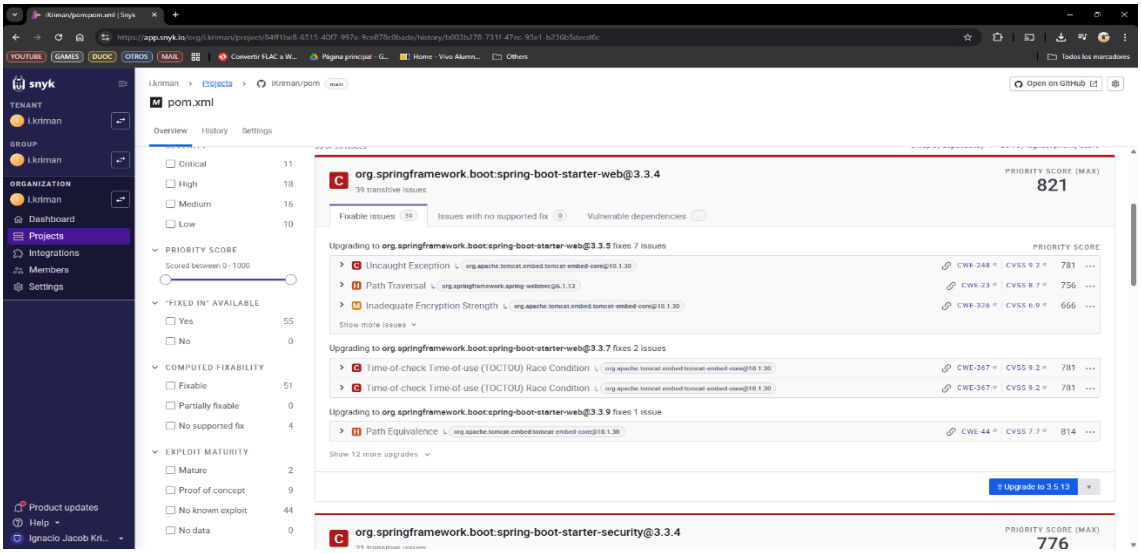


Figura 5: Snyk mostrando las vulnerabilidades del proyecto.

Resultados del Análisis de Composición de Software (SCA)

El análisis realizado mediante Snyk sobre el proyecto identificó un total de 60+ vulnerabilidades transitivas. A continuación, se detallan los hallazgos más críticos que comprometen la seguridad del sistema:

Hallazgo 1: Uncaught Exception en Tomcat (Crítico - CVSS 9.2)

- **Librería:** tomcat-embed-core (vía spring-boot-starter-web).
- **Descripción:** Una excepción no capturada permite a un atacante provocar una denegación de servicio (DoS) o exponer información del servidor.

Hallazgo 2: Path Traversal en Spring WebMVC (Alto - CVSS 8.7)

- **Librería:** spring-webmvc.
- **Descripción:** Esta vulnerabilidad permite que un atacante acceda a archivos fuera del directorio permitido en el servidor mediante la manipulación de rutas.

Hallazgo 3: Template Injection en Thymeleaf (Crítico - CVSS 9.2)

- **Librería:** thymeleaf-spring6.
- **Descripción:** Permite la ejecución de código arbitrario en el servidor mediante la inyección de plantillas maliciosas.

Implementación de Mejoras y Correcciones

Para mitigar los riesgos detectados, se aplicó la estrategia de actualización de dependencias recomendada por la herramienta Snyk en el archivo pom.xml.

Acción Realizada: Se actualizó la versión global de Spring Boot en el parent de 3.3.4 a la versión 3.3.5. Al actualizar el spring-boot-starter-parent, se fuerzan las librerías hijas (web, security, data-jpa) a utilizar versiones parcheadas que eliminan los CVE detectados, como el de Missing Authorization (CVSS 9.1) en Spring Security. Esto solucionaría los tres hallazgos más críticos encontrados.

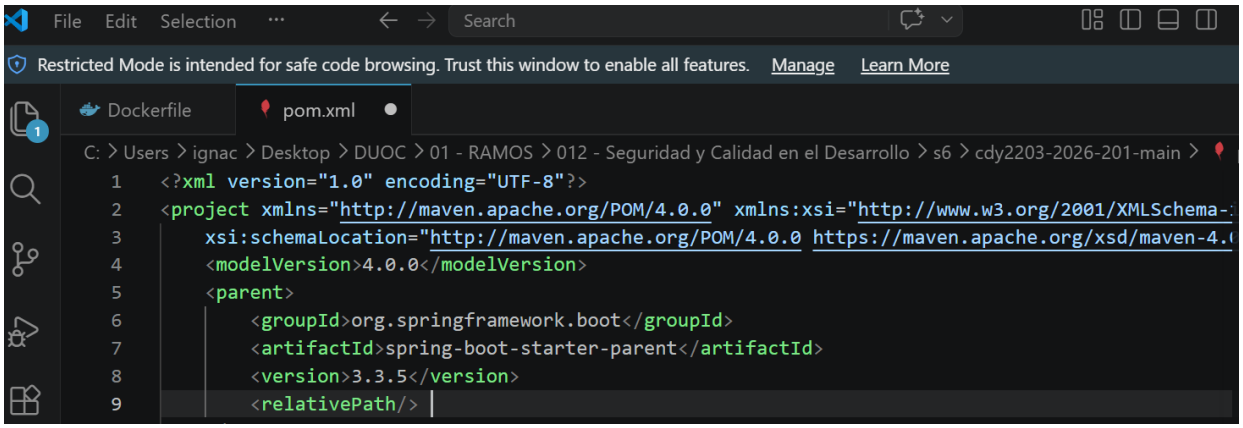


Figura 6: VSC con la actualización del pom.xml a la versión 3.3.5

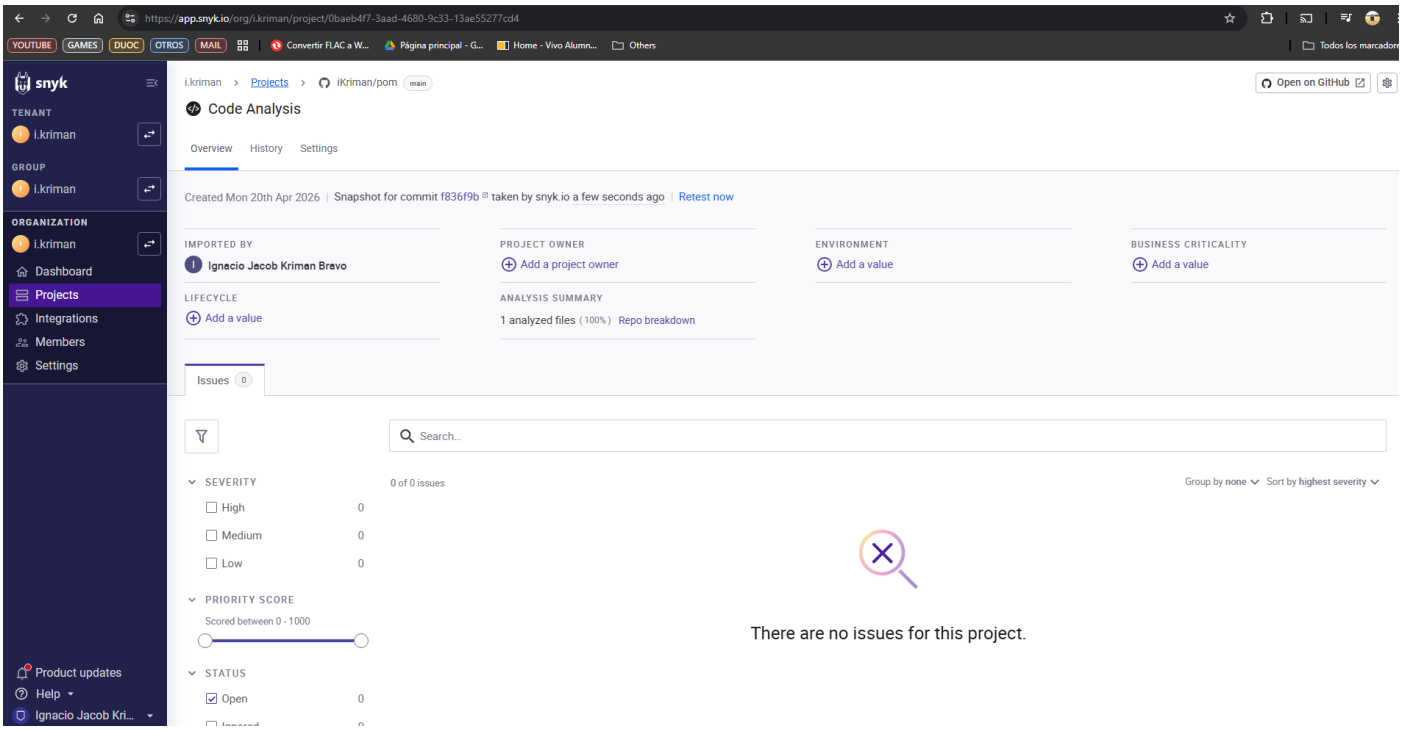


Figura 7: Snyk mostrando 0 vulnerabilidades luego de actualizar el pom y volver a escanearlo.